

PARTE A — LISTA, PILHA E FILA

1) Implemente a função que insere um nó no final de uma lista encadeada.

```
void inserirFinal(No **lista, int valor);
```

2) Remova a primeira ocorrência de um valor na lista.

```
void remover(No **lista, int valor);
```

3) Retorne a quantidade de nós da lista.

```
int contar(No *lista);
```

4) Retorne 1 se o valor existir, 0 caso contrário.

```
int buscar(No *lista, int valor);
```

5) Implemente inserção em uma pilha.

```
void push(No **pilha, int valor);
```

6) Remova e retorne o topo da pilha.

```
int pop(No **pilha);
```

7) Inserir elemento no final da fila.

```
void enqueue(No **fila, int valor);
```

8) Remover elemento do início da fila.

```
int dequeue(No **fila);
```

9) Inverta uma lista usando uma pilha.

```
void inverterLista(No **lista);
```

10) Transfira elementos de uma fila para uma lista.

```
void filaParaLista(No **fila, No **lista);
```

PARTE B — ÁRVORES BINÁRIAS DE BUSCA

11) Inserção com contagem de comparações.

```
int inserirCont(No **raiz, int valor);
```

12) Busca exibindo o caminho percorrido.

```
int buscarCaminho(No *raiz, int valor);
```

13) Retorne o menor valor da árvore.

```
int menor(No *raiz);
```

14) Retorne o maior valor da árvore.

```
int maior(No *raiz);
```

15) Soma dos nós maiores que X.

```
int somaMaiores(No *raiz, int x);
```

16) Contar nós no intervalo [a, b].

```
int contarIntervalo(No *raiz, int a, int b);
```

17) Retorne o nível de um nó.

```
int nivel(No *raiz, int valor);
```

18) Verifique se a árvore é estritamente binária.

```
int ehEstrita(No *raiz);
```

19) Espelhar a árvore.

```
void espelhar(No *raiz);
```

20) Verificar se duas árvores são iguais.

```
int iguais(No *a, No *b);
```

21) Remoção considerando nó folha.

```
No* remover(No *raiz, int valor);
```

22) Remoção de nó com um filho.

23) Remoção completa (todos os casos).

24) Encontrar sucessor.

No* sucessor(No *raiz);

25) Verificar se é BST.

int ehBST(No *raiz);

CRITÉRIOS DE AVALIAÇÃO

- Correção lógica do algoritmo (40%)
- Uso correto de ponteiros e memória (20%)
- Estrutura e organização do código (15%)
- Tratamento de casos especiais (15%)
- Clareza e legibilidade (10%)